

Operating Systems

Rowan University

Overview

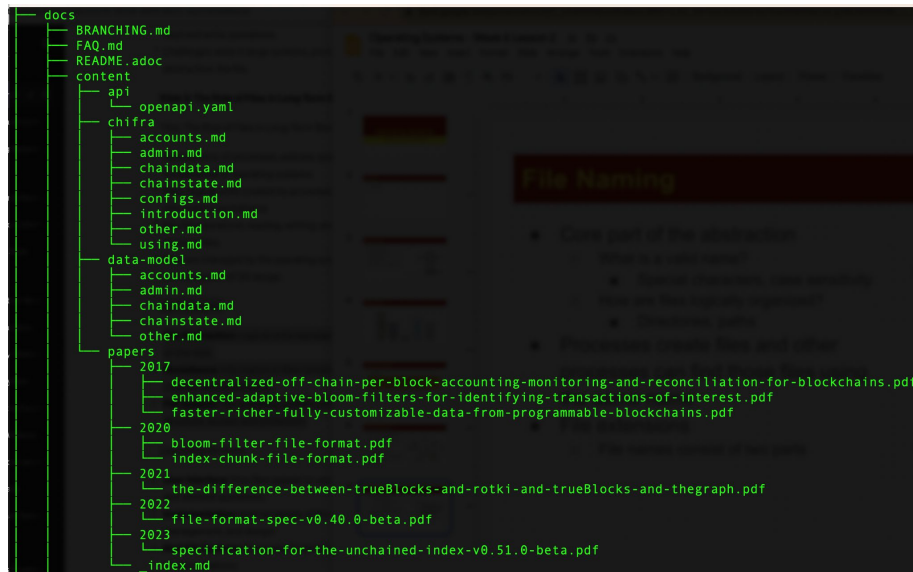
1. File Systems
 - a. Files
 - b. Directories
2. File System Implementation

Long Term Storage

- All computer applications face three critical challenges in managing information:
 1. Limited storage capacity in physical RAM during process execution
 2. Information loss upon process termination
 3. The need for concurrent access by multiple processes
- Magnetic disks and solid-state drives (SSDs) are key solutions for long-term storage.
- Challenges arise in large systems, prompting the need for a new abstraction: *the file*

OS File Abstraction

- **Files** - logical units created by processes
 - model the disk and ensure information *persistence*
- **Key file operations:**
 - Reading
 - Writing
 - Create and manage
- **File System** - Files are managed by the operating system through the *file system*



File Naming

- Core part of the abstraction
 - What is a valid name?
 - Special characters, case sensitivity
 - How are files logically organized?
 - Directories, paths
- Processes create files and other processes can find those files using the same name
- **File extensions**
 - File names consist of two parts
 - Name.extension
 - OS understand the file extension meaning
 - `file` command reports type of file

Figure 4-1

Extension	Meaning
.bak	Backup file
.c	C source program
.gif	Compuserve Graphical Interchange Format image
.html	World Wide Web HyperText Markup Language document
.jpg	Still picture encoded with the JPEG standard
.mp3	Music encoded in MPEG layer 3 audio format
.mpg	Movie encoded with the MPEG standard
.o	Object file (compiler output, not yet linked)
.pdf	Portable Document Format file
.ps	PostScript file
.tex	Input for the TEX formatting program
.txt	General text file
.zip	Compressed archive

Some typical file extensions.

File Structure

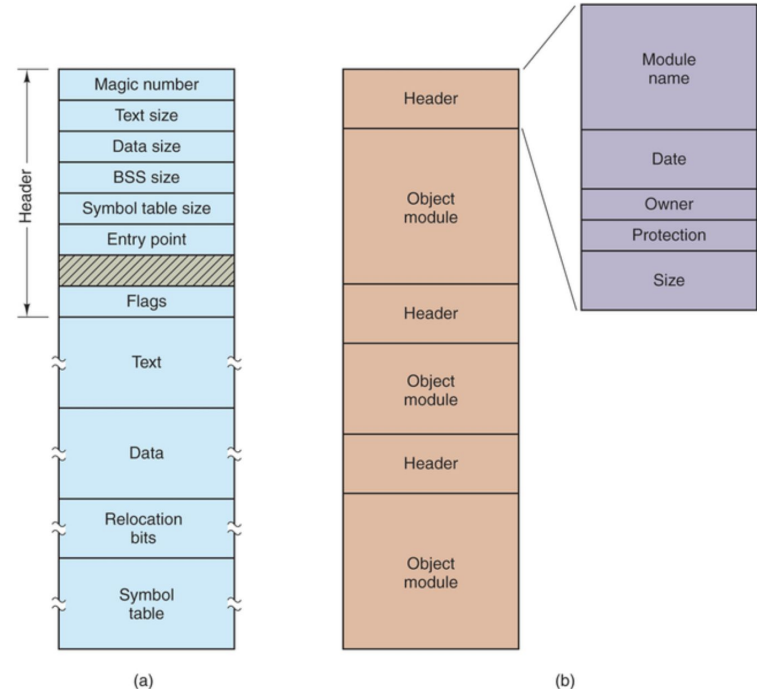
- OS regards all files as simply bytes
- The bytes of a file are structured
- Structure depends on:
 - The file type (e.g. .zip, .exe)
 - The program that will use the file
- The OS can determine the type of file for some files

Figure 4-4

Command	Result
file README.txt	UTF-8 Unicode text
file hjb.sh	POSIX shell script, ASCII text executable
file Makefile	makefile script, ASCII text
file /usr/bin/less	symbolic link to /bin/less
file /bin/	directory
file /bin/less	ELF 64-bit LSB shared object, x86-64 [...more information...]

Finding out file types.

Figure 4-3



(a) An executable file. (b) An archive.

File Access

- Access files on a byte by byte level
- **read** operation gives the position in the file to start reading at
- **seek** sets the current position in the file
- *File pointer* - reference to the start of a file, keeps track of the location within the file while reading it

```
>>> f = open('workfile', 'rb+')
>>> f.write(b'0123456789abcdef')
16
>>> f.seek(5)      # Go to the 6th byte in the file
5
>>> f.read(1)
b'5'
>>> f.seek(-3, 2)  # Go to the 3rd byte before the end
13
>>> f.read(1)
b'd'
```

File Attributes, Metadata

- Protection Attributes
 - Protection: Specifies who can access the file and in what manner.
 - Password: Some systems require a password for file access.
- Flags (Bits or Short Fields)
 - Read-only, Hidden, System, Archive: Control file properties.
 - ASCII/Binary: Differentiate between ASCII and binary files.
 - Random Access, Temporary, Lock Flags: Manage access and file behavior.
- Record-related Attributes
 - Record Length, Key Position, Key Length: Relevant for files with lookupable records.
- Time-related Attributes
 - Creation Time, Last Access Time, Last Change Time: Track file temporal aspects.
- Size Attributes
 - Current Size: Present file size.
 - Maximum Size: Specifies the maximum size a file may reach.

```
Cloud-Builder-Community node_modules Picrostic
● (base) mghen@Michaels-MacBook-Pro-2 blocksec % exiftool package.json
ExifTool Version Number      : 12.63
File Name                    : package.json
Directory                    : .
File Size                    : 55 bytes
File Modification Date/Time   : 2021:03:27 14:37:49-04:00
File Access Date/Time        : 2021:07:30 19:23:12-04:00
File Inode Change Date/Time   : 2021:07:30 20:19:32-04:00
File Permissions              : -rw-r--r--
File Type                    : JSON
File Type Extension          : json
MIME Type                    : application/json
Dev Dependencies Hardhat      : ^2.1.2
```


File Operations

- Create
 - Announce file creation and set initial attributes.
- Delete
 - Release disk space by deleting a file.
- Open
 - Fetch file attributes and disk addresses for rapid access.
- Close
 - Release internal table space by closing a file.
- Read
 - Retrieve data from a file; specify bytes and provide a buffer.
- Write
 - Add data to a file, usually at the current position; increases file size.
- Append
 - Restricted Write; adds data only to the end of the file.
- Seek
 - Reposition file pointer for random-access files; enables specific positions.
- Attributes Retrieval
 - Obtain file attributes for process use.
- Attributes Modification
 - Change user-settable attributes after file creation.
- Rename
 - Efficiently rename a file, avoiding time-consuming copying and deleting.

Figure 4-6

```
/* File copy program. Error checking and reporting is minimal. */

#include <sys/types.h> /* include necessary header files */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]); /* ANSI prototype */

#define BUF_SIZE 4096 /* use a buffer size of 4096 bytes */
#define OUTPUT_MODE 0700 /* protection bits for output file */

int main(int argc, char *argv[])
{
    int in_fd, out_fd, rd_count, wt_count;
    char buffer[BUF_SIZE];

    if (argc != 3) exit(1); /* syntax error if argc is not 3 */

    /* Open the input file and create the output file */
    in_fd = open(argv[1], O_RDONLY); /* open the source file */
    if (in_fd < 0) exit(2); /* if it cannot be opened, exit */
    out_fd = creat(argv[2], OUTPUT_MODE); /* create the destination file */
    if (out_fd < 0) exit(3); /* if it cannot be created, exit */

    /* Copy loop */
    while (TRUE) {
        rd_count = read(in_fd, buffer, BUF_SIZE); /* read a block of data */
        if (rd_count <= 0) break; /* if end of file or error, exit loop */
        wt_count = write(out_fd, buffer, rd_count); /* write data */
        if (wt_count <= 0) exit(4); /* wt_count <= 0 is an error */
    }

    /* Close the files */
    close(in_fd);
    close(out_fd);
    if (rd_count == 0) /* no error on last read */
        exit(0);
    else
        exit(5); /* error on last read */
}
```

Directories

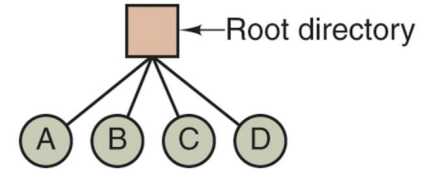
- Single-Level Directory Systems

- Simplest form: one directory for all files, often termed the root directory
- Advantages: Simplicity and quick file location due to a single lookup location

- Hierarchical Directory Systems

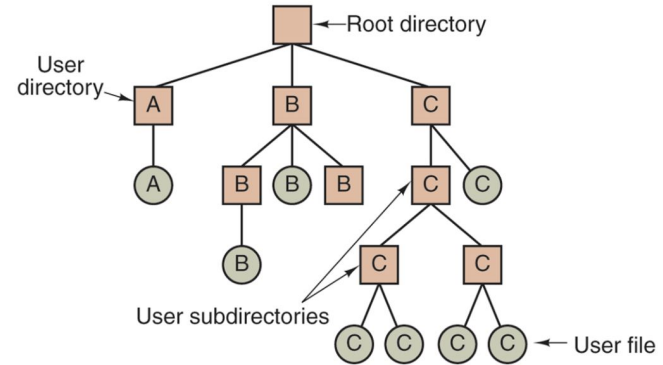
- Adequate for modern users with thousands of files
- Hierarchy: A tree of directories; allows grouping related files naturally
- ex : `etc` - configuration files

Figure 4-7



A single-level directory system containing four files.

Figure 4-8

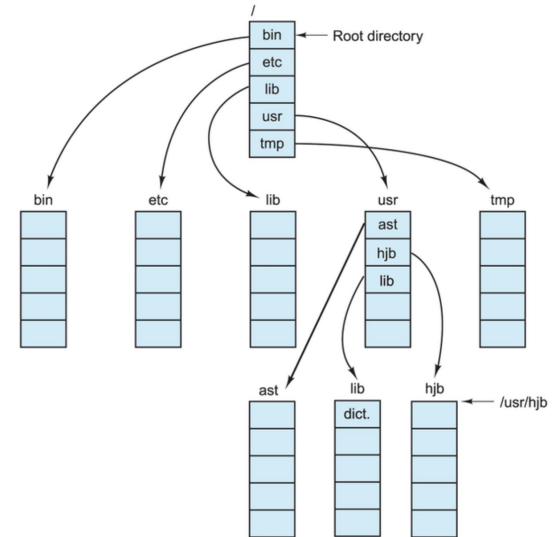


A hierarchical directory system.

Path Names

- Absolute Path Names
 - Given path from the root directory to the file
 - Example: `/usr/ast/mailbox`
- Relative Path Names
 - Path names *relative* to the current working directory
- Working Directory
 - Each process has its own working directory
 - Can be changed at any time
- Special Entries in Directories
 - “.” (dot): Refers to the current directory.
 - “..” (dotdot): Refers to the parent directory
 - “~”: Refers to the user home directory

Figure 4-9



A UNIX directory tree.

Directory Operations

- **Creation**
 - New directory created, initially containing dot and dotdot
- **Deletion**
 - Deletes an empty directory; dot and dotdot considered
- **Open**
 - Directory opened for reading its contents
- **Read**
 - Returns the next entry in an open directory
- **Close**
 - Closes a previously opened directory
- **Rename**
 - Renames a directory
- **Linking:**
 - Creates a hard link to an existing file in a different directory
- **Unlinking:**
 - Removes a directory entry; if file in multiple directories, only specified path is removed

Symbolic Link - A small file that names another file so when it is referenced the other file is used

Advantage is they can cross disk boundaries, even reference remote file systems

Root and Home Directory

```
(base) mghen@michaels-mbp-2 / % tree -L 1
```

```
.  
├── Applications  
├── Library  
├── System  
├── Users  
├── Volumes  
├── bin  
├── cores  
├── dev  
├── etc -> private/etc  
├── home -> /System/Volumes/Data/home  
├── opt  
├── private  
├── sbin  
├── tmp -> private/tmp  
├── usr  
└── var -> private/var
```

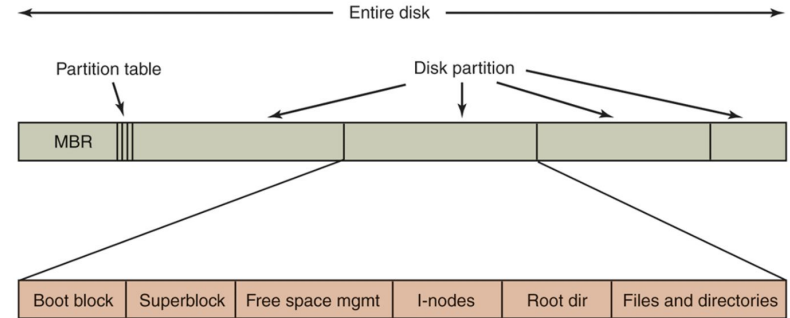
```
(base) mghen@michaels-mbp-2 ~ % tree -L 1
```

```
.  
├── Applications  
├── Desktop  
├── Desktop.sql  
├── Documents  
├── Downloads  
├── Library  
├── Movies  
├── Music  
├── Pictures  
├── Postman  
└── Public
```

File System Implementation

- Master Boot Record (MBR)
 - Sector 0 of the disk, used to boot the computer
 - Contains the partition table with starting and ending addresses of each partition
 - One partition marked as active for booting
- Booting Process
 - BIOS reads and executes the MBR
 - MBR locates the active partition, reads its boot block, and executes it
 - Boot block loads the operating system in that partition

Figure 4-10



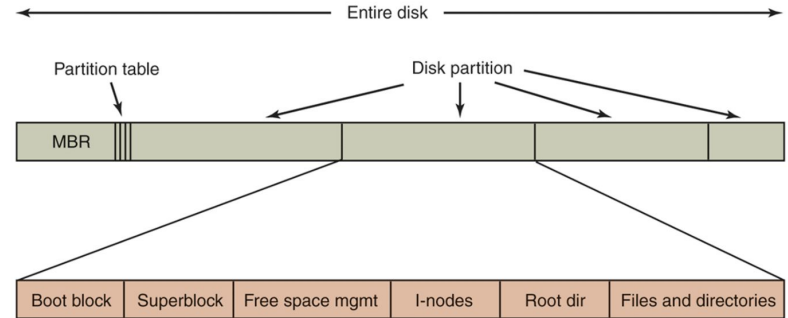
A possible file-system layout.

Master Boot Record

- File System Layout:

- *Superblock*
 - Contains key parameters about the file system
 - Read into memory during boot or first file system access
 - Includes magic number, file-system size, and administrative information
- *Free Space Management*
 - Information about free blocks, often in the form of a bitmap or list of pointers
- *I-nodes*
 - Data structures, one per file, containing information about each file
- *Root Directory*
 - Contains the top of the file-system tree
- *Files and Directories*
 - The remainder of the disk contains all other directories and files

Figure 4-10

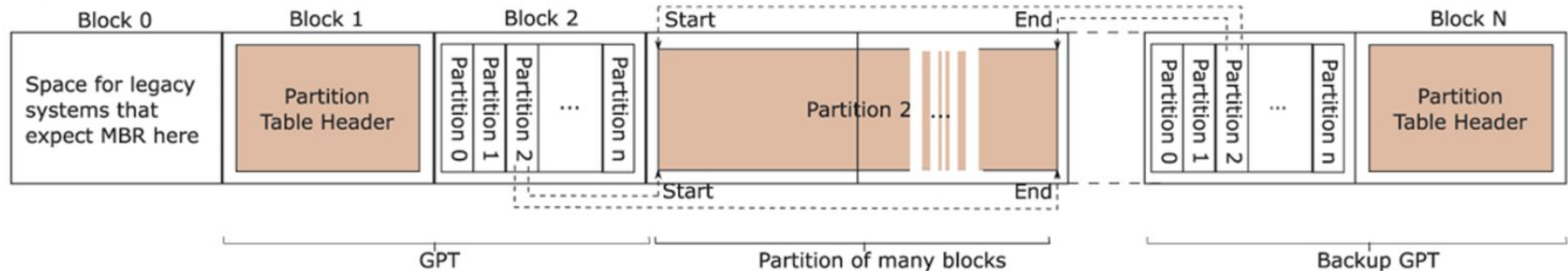


A possible file-system layout.

Unified Extensible Firmware Interface (UEFI)

- Partition table at block 1
- GUID Partition Table
 - Information about partition location
 - Uses globally unique identifiers (GUID)
 - Backs this up in the last block
- UEFI Boot Process (Fig. 4-11):
 - UEFI firmware reads GPT to locate partitions
 - Supports at least FAT file system types, with an EFI system partition (ESP) containing the bootloader
 - Allows the boot process to use a proper file system with programs, configuration files, etc

Figure 4-11



Layout for UEFI with partition table.

The UEFI Boot Process

UEFI uses a Boot Manager in the EFI System Partition (ESP) to locate and launch the operating system using configuration files rather than raw disk sectors.

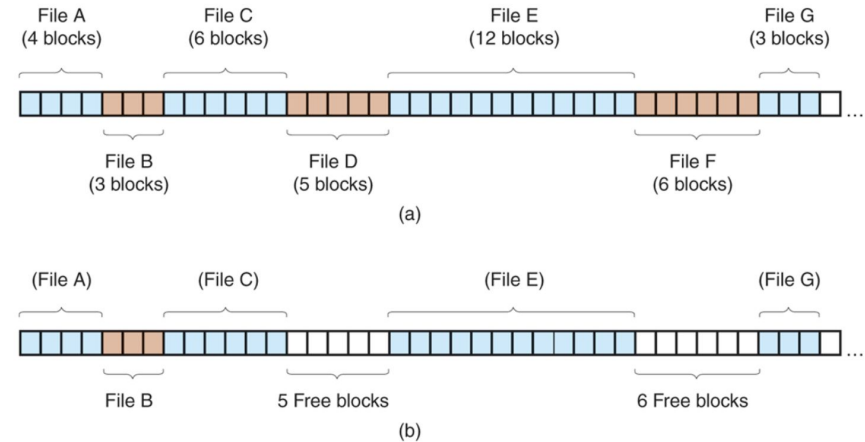
Execution Phases

- SEC (Security): The initial power-on phase that initializes the CPU and temporary memory.
- PEI (Pre-EFI Initialization): Identifies and initializes critical hardware, such as system RAM.
- DXE (Driver Execution Environment): Loads hardware drivers (disk, keyboard, video) to enable system communication.
- BDS (Boot Device Selection): Uses the Boot Manager to find the OS loader (e.g., bootmgfw.efi) and hands over control.

File System Implementations

- **Contiguous Allocation**
- **Advantage**
 - Simple
 - Read performance is good
- **Drawback**
 - Disk fragmentation
- **Challenge:**
 - Final file size must be known
 - Not a good user experience

Figure 4-12

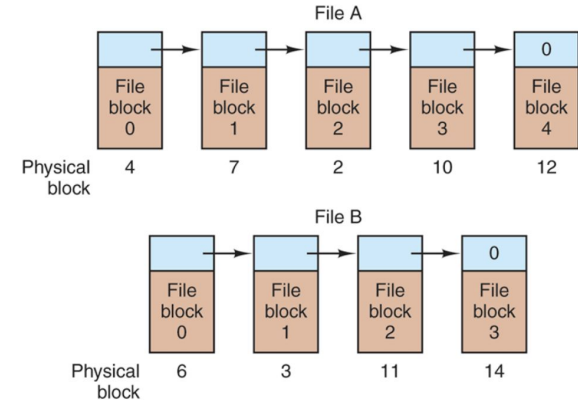


(a) Contiguous allocation of disk space for seven files. (b) The state of the disk after files D and F have been removed.

File System Implementation

- **Linked-List Allocation**
- Each file stored as a linked list disk blocks
- **Advantage**
 - No disk fragmentation
 - Efficient directory entry, store first block
- **Disadvantage**
 - Slow random access
 - Extra overhead for getting data from multiple disk blocks

Figure 4-13

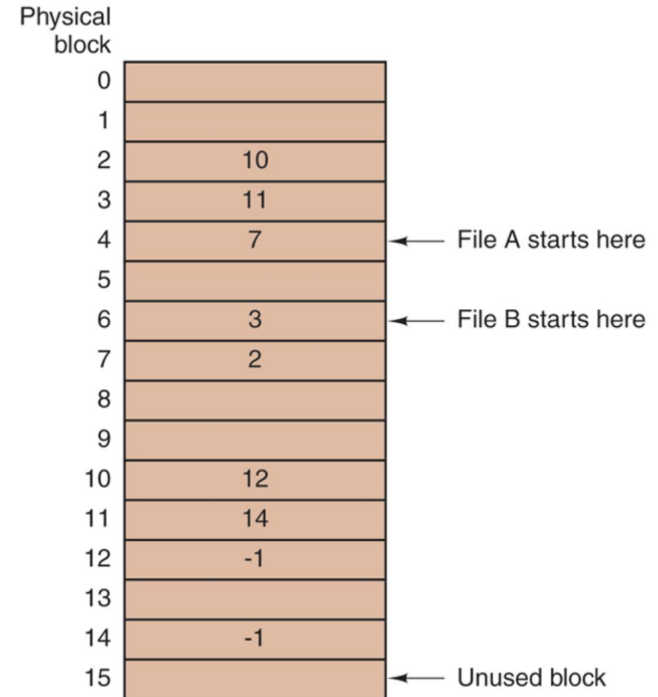


Storing a file as a linked list of disk blocks.

File System Implementation

- **Linked-List using Allocation Table**
- **FAT** - File Allocation Table
- Moves pointers from disk blocks to memory
- **Advantages**
 - Frees the block to be used in full
 - Faster random access, chain in memory
 - Directory entries are a single number
- **Disadvantages**
 - Uses memory
 - Table size increases as disk size increases

Figure 4-14

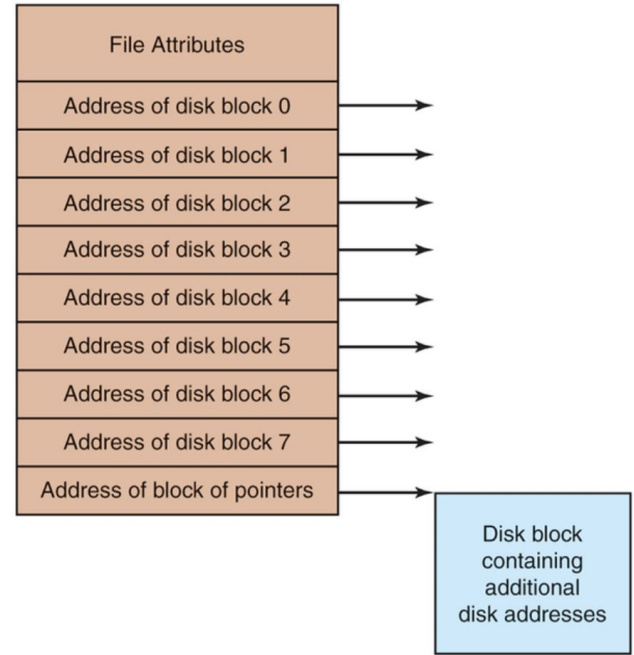


Linked-list allocation using a file-allocation table in main memory.

File System Implementation

- **I-Nodes**
- Associates attributes and disk addresses with each file
- Disk address that points to an index for the file
- I-node is only in memory when the file is open
- There's a maximum number of files that can be open

Figure 4-15



An example i-node.

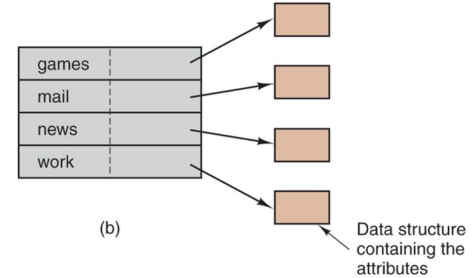
Implementing Directories

- Operating systems use path names to locate directory entries on the disk, providing information to find disk blocks
- Directory Entry Structure
 - Option 1
 - Simple design with fixed-size entries
 - Each entry contains a file name, attributes, and disk addresses
 - Option 2
 - For systems using i-nodes
 - Directory entry has a file name and i-node number
 - Attributes are stored in the i-node

Figure 4-16

games	attributes
mail	attributes
news	attributes
work	attributes

(a)

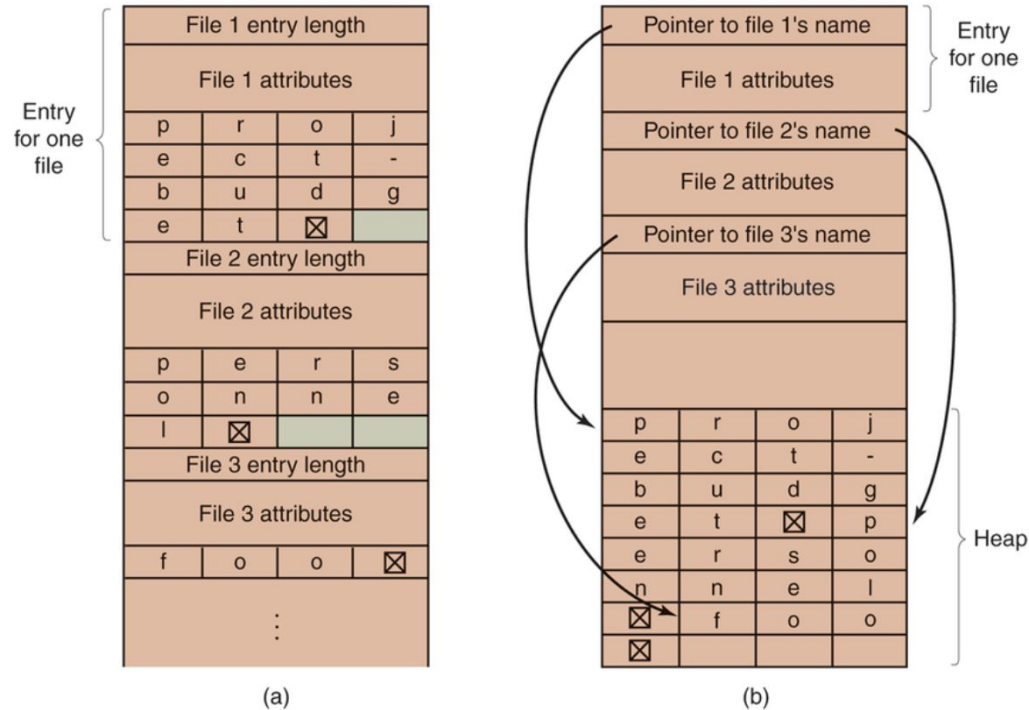


(b)

(a) A simple directory containing fixed-size entries with the disk addresses and attributes in the directory entry. (b) A directory in which each entry just refers to an i-node.

Implementing Directories

Figure 4-17



Two ways of handling long file names in a directory. (a) In-line. (b) In a heap.